

Lección 0.2 - Python, NumPy, PyTorch

Lección 0.2 — Python, NumPy, PyTorch: Las Herramientas

Tags: #fundamentos #python #pytorch #numpy

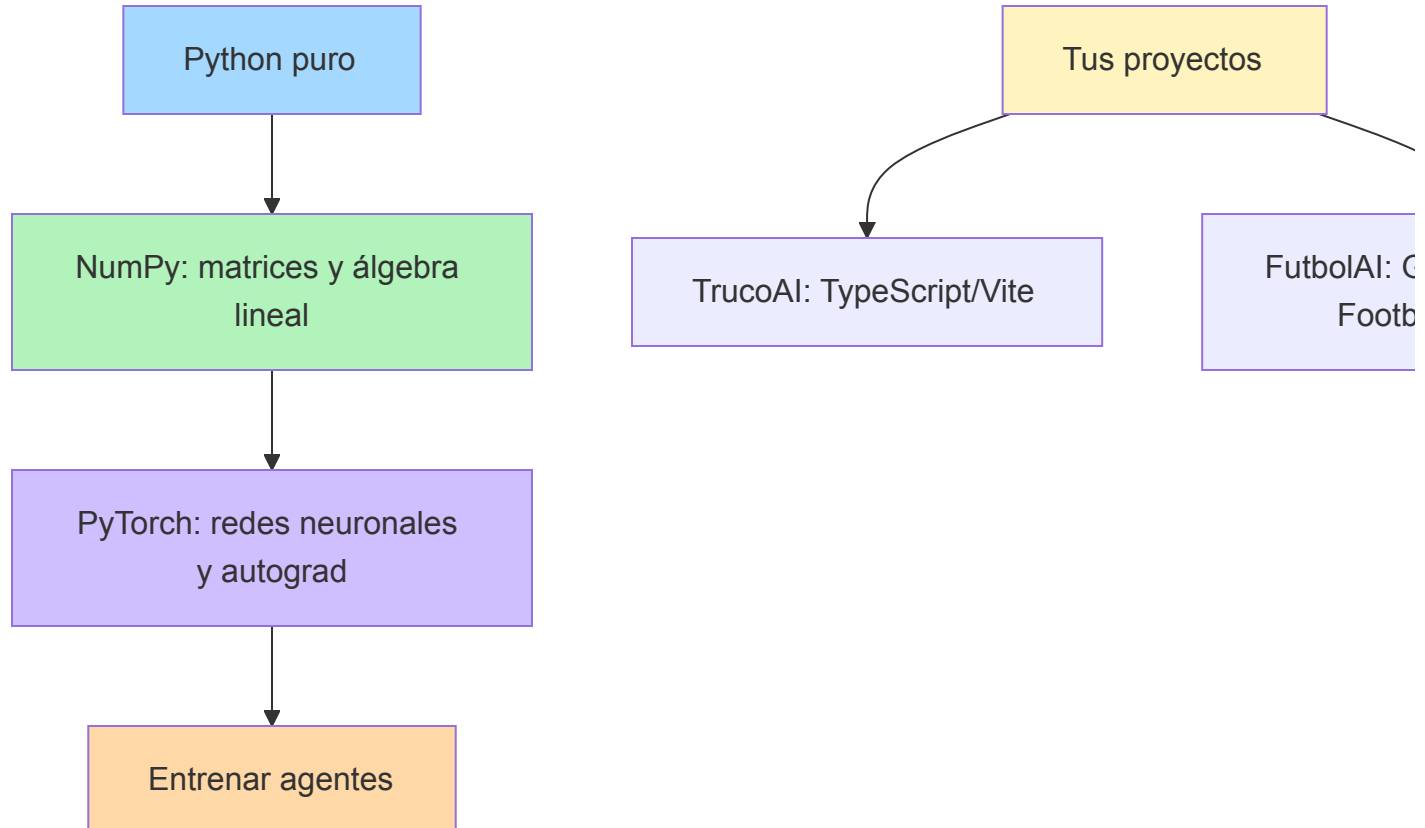
Duración estimada: 4 días

Prerrequisitos: Lección 0.1

Objetivo

Pasar de "sé lo básico de Python" a poder implementar los algoritmos de los papers (CFR, redes neuronales, simulated annealing) por tu cuenta.

Stack tecnológico



1. Python — Lo que realmente vas a usar

No necesitás ser experto. Necesitás entender **estos** patrones:

1.1 Tipos de datos para ML

```
# Listas → colecciones de cartas, historial de jugadas
mano = [3, 2, 7] # valores de tus cartas
historial = [(0, 'jugó_3'), (1, 'cantó_truco'), ...]

# Arrays de NumPy → vectores de estado para la red neuronal
import numpy as np
estado = np.array([3.0, 2.0, 7.0, 12.0, 5.0, 1.0, 0.0])

# Tensores de PyTorch → lo que alimenta la red
import torch
entrada = torch.tensor([estado], dtype=torch.float32)
```

1.2 El patrón "entrenar un modelo"

```
for episodio in range(10000):
    estado = ambiente.reset()
    done = False
    while not done:
        accion = agente.seleccionar_accion(estado)
        nuevo_estado, recompensa, done = ambiente.step(accion)
        agente.aprender(estado, accion, recompensa, nuevo_estado)
        estado = nuevo_estado
```

Este loop aparece en TODOS los papers de RL. Es el corazón.

1.3 Clases para agentes

```
class AgenteTruco:
    """Un agente que juega al Truco."""

    def __init__(self, nombre):
        self.nombre = nombre
        self.mano = []
        self.score = 0

    def recibir_cartas(self, cartas):
        self.mano = cartas
```

```

def jugar_carta(self, ronda, historial):
    # Lógica de selección
    mejor_carta = self.evaluar_opciones(ronda, historial)
    self.mano.remove(mejor_carta)
    return mejor_carta

def evaluar_opciones(self, ronda, historial):
    # Aquí va CFR, red neuronal, o random
    return self.mano[0] # placeholder

```

2. NumPy — Álgebra lineal rápida

2.1 Crear arrays

```

import numpy as np

# Vector de estado del Truco (simplificado)
estado = np.array([3.0, 2.0, 7.0,          # tus cartas
                  0.0, 1.0, 0.0,          # cartas jugadas esta ronda (one-
hot)
                  12.0, 5.0,              # scores
                  1.0])                  # ¿truco activo?

# Matriz de policy (probabilidad de cada acción)
# 10 acciones posibles, 9 features de entrada
W = np.random.randn(9, 10) * 0.01

# Dot product → scores por acción
scores = estado @ W # o np.dot(estado, W)
accion_elegida = np.argmax(scores)

```

2.2 Operaciones clave

```

# Promedio de recompensas en un batch
recompensas = np.array([1, -1, 3, -1, 1])
promedio = np.mean(recompensas)          # 0.6
desvio = np.std(recompensas)             # 1.36

# Normalizar estado (importantísimo para entrenar redes)
def normalizar(vector):
    return (vector - np.mean(vector)) / (np.std(vector) + 1e-8)

```

```
# Muestreo de acciones según probabilidades
probs = np.array([0.2, 0.5, 0.3]) # probabilidad de cada carta
accion = np.random.choice(3, p=probs) # elige según las prob
```

3. PyTorch — Redes neuronales

3.1 El concepto en 5 líneas

```
import torch
import torch.nn as nn
import torch.optim as optim

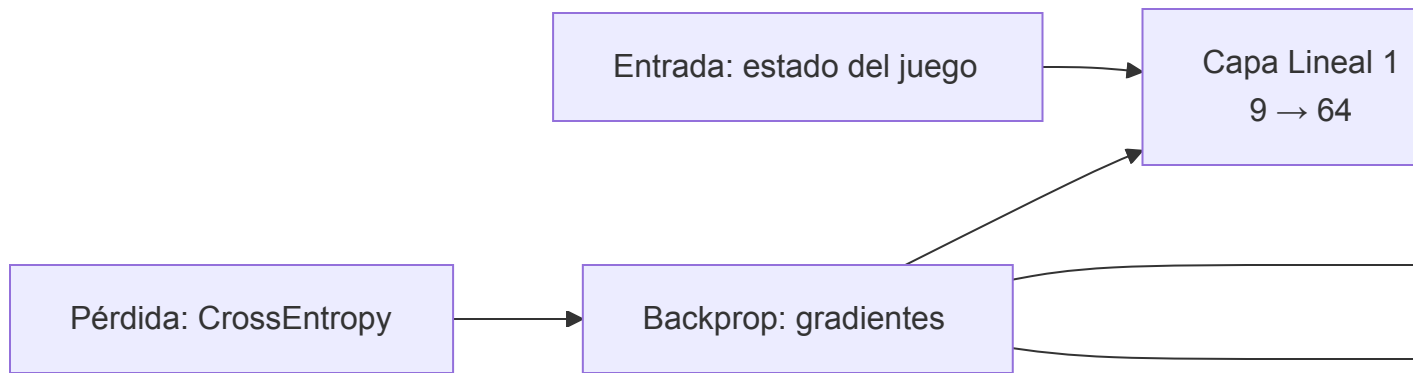
# 1. Definir la red → una clase
class RedTruco(nn.Module):
    def __init__(self, entrada=9, salida=4):
        super().__init__()
        self.capa1 = nn.Linear(entrada, 64)
        self.capa2 = nn.Linear(64, 64)
        self.salida = nn.Linear(64, salida)

    def forward(self, x):
        x = torch.relu(self.capa1(x))
        x = torch.relu(self.capa2(x))
        return self.salida(x) # logits (sin softmax)

# 2. Instanciar
red = RedTruco()
optimizador = optim.Adam(red.parameters(), lr=0.001)

# 3. Un paso de entrenamiento
estado = torch.tensor([[3.0, 2.0, 7.0, 0.0, 1.0, 0.0, 12.0, 5.0, 1.0]])
prediccion = red(estado) # forward
# ... calcular pérdida, backprop...
```

3.2 El gráfico de computación



3.3 Autograd — La magia

PyTorch calcula automáticamente las derivadas (gradientes) de cada operación. No necesitás saber cálculo para entrenar una red:

```

x = torch.tensor([3.0, 2.0, 7.0], requires_grad=True)
y = (x ** 2).sum() # y = 32 + 22 + 72 = 62
y.backward()      # calcula dy/dx para cada x
print(x.grad)     # tensor([6., 4., 14.]) → 2*x

```

4. Conexión con los papers

Deep CFR (Brown et al. 2019)

```

# El paper Deep CFR usa DOS redes:
class DeepCFR(nn.Module):
    def __init__(self):
        super().__init__()
        self.value_net = RedTruco(9, 1) # Valor de cada estado
        self.policy_net = RedTruco(9, 4) # Política óptima

    def forward(self, estado):
        valor = self.value_net(estado)
        policy = torch.softmax(self.policy_net(estado), dim=-1)
        return policy, valor

```

Single Deep CFR (SD-CFR)

```

# SD-CFR simplifica: una sola red
class SDCFR(nn.Module):
    def __init__(self):

```

```
super().__init__()
# Una sola red que hace todo
self.red = RedTruco(9, 5) # 4 acciones + 1 valor
```

Policy Gradient (Chrustowski & Duch 2025)

```
# REINFORCE con experience replay
class PolicyGradient:
    def __init__(self):
        self.policy = RedTruco(9, 4)
        self.memoria = [] # experience replay buffer

    def train_step(self, estados, acciones, recompensas):
        log_probs = torch.log_softmax(self.policy(estados), dim=-1)
        # REINFORCE loss:  $-\log(\pi(a|s)) \times R$ 
        loss = -(log_probs * recompensas).mean()
        loss.backward()
```

5. Configuración para tu Mac

```
# Ya tenés Python 3.9 (lo verificamos)
python3 --version

# Lo que instalaste ya
python3 -m pip install pypdf

# Lo que vas a necesitar
python3 -m pip install numpy matplotlib torch gym gfootball
```

Ejercicios

Ejercicio 1 — Tu primera red

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np

# Dataset sintético: estado → mejor acción
```

```

# Suponemos que la acción correcta es jugar la carta más alta
def generar_datos(n=1000):
    X, y = [], []
    for _ in range(n):
        mano = np.random.choice(range(1, 13), 3, replace=False)
        mejor = np.argmax(mano) # jugar la más alta
        X.append(mano)
        y.append(mejor)
    return torch.tensor(X, dtype=torch.float32), torch.tensor(y)

X, y = generar_datos()
modelo = nn.Sequential(
    nn.Linear(3, 16),
    nn.ReLU(),
    nn.Linear(16, 3)
)
optimizador = optim.Adam(modelo.parameters(), lr=0.01)
perdida = nn.CrossEntropyLoss()

for epoca in range(500):
    pred = modelo(X)
    loss = perdida(pred, y)
    optimizador.zero_grad()
    loss.backward()
    optimizador.step()

# Probar
test = torch.tensor([[1.0, 12.0, 7.0]])
pred = modelo(test)
accion = torch.argmax(pred).item()
print(f"Con cartas [1, 12, 7], juego la posición {accion}")
# Esperado: posición 1 (el 12)

```

Ejercicio 2 — Experience Replay Buffer

Implementá un buffer circular que almacene las últimas 10,000 experiencias (estado, acción, recompensa, próximo_estado) y permita samplear batches aleatorios. Esto se usa en Deep CFR y en DQN.

Recursos

- **PyTorch Tutorial oficial:** pytorch.org/tutorials — "Learn the Basics"
- **Video:** "PyTorch in 60 Minutes" (YouTube)

- **Video:** "NumPy Crash Course" por freeCodeCamp
 - **Notebook:** Colab con el ejercicio resuelto (link pronto)
-



Checkpoint

- ☐ Podés crear un vector de estado de Truco con NumPy
 - ☐ Definiste una red neuronal de 3 capas en PyTorch
 - ☐ Entendés qué hace `backward()` y por qué existe
 - ☐ Implementaste el loop de entrenamiento
 - ☐ Conectaste los conceptos con los papers (Deep CFR, SD-CFR, REINFORCE)
-

Siguiente: [Lección 0.3 — TypeScript y Vite para TrucoAI](#)